Zixir Language Complete Guide

A Comprehensive Guide for Beginners and Developers

Version 1.0 - Production Ready

Table of Contents

---

1. Introduction

What is Zixir?

Zixir is a modern, expression-oriented programming language designed specifically for **AI automation and development workflows**. Unlike traditional languages that require you to write dozens of lines of boilerplate code, Zixir lets you express complex operations in just a few lines.

The Three-Tier Architecture

Zixir is unique because it combines three powerful technologies:

1. **Elixir** - The orchestrator that manages everything
2. **Zig** - The high-performance engine for math and data operations (10-100x faster)
3. **Python** - Access to the world's largest library ecosystem

Think of it like a supercar: Elixir is the driver, Zig is the engine, and Python is the toolkit in the trunk.

Why Zixir for AI Automation?

Before Zixir:

```
# Python only - verbose and slow for math
import numpy as np
import json

data = [1.0, 2.0, 3.0, 4.0, 5.0]
mean = np.mean(data)
std = np.std(data)
result = json.dumps({"mean": mean, "std": std})
```

With Zixir:

```
let data = [1.0, 2.0, 3.0, 4.0, 5.0]
{"mean": engine.list_mean(data), "std": engine.list_std(data)}
```

Who This Guide Is For

- **Complete beginners**: No programming experience required
- **Python/JavaScript developers**: Learn Zixir's unique features
- **AI/ML engineers**: Build automation workflows faster

What You'll Learn

By the end of this guide, you'll be able to:

- Write complete Zixir programs
- Process data using high-performance engine operations
- Call Python libraries seamlessly
- Build real AI automation workflows
- Debug and optimize your code

---

2. Getting Started

Installation

Step 1: Install Elixir and Erlang

macOS:

```
brew install elixir
```

Ubuntu/Debian:

```
wget https://packages.erlang-solutions.com/erlang-solutions_2.0_all.deb
sudo dpkg -i erlang-solutions_2.0_all.deb
sudo apt-get update
sudo apt-get install esl-erlang elixir
```

Windows: Download and run the installer from: https://elixir-lang.org/install.html

Verify installation:

```
elixir --version
```

Expected output (version numbers may vary):

```
Erlang/OTP 25 [erts-13.0] [source] [64-bit] [smp:8:8]
Elixir 1.14.0 (compiled with Erlang/OTP 25)
```

Step 2: Install Python (Optional, for FFI)

Zixir can call Python libraries, but Python is optional if you only use engine operations.

macOS:

```
brew install python
```

Ubuntu/Debian:

```
sudo apt-get install python3 python3-pip
```

Windows: Download from https://www.python.org/downloads/

Verify:

```
python3 --version
```

Step 3: Install Zixir

```
# Clone the repository
git clone https://github.com/Zixir-lang/Zixir.git
cd Zixir

# Install dependencies
mix deps.get

# Compile the project
mix compile

# Run tests (optional, takes a few minutes)
mix test
```

Step 4: Verify Installation

```
# Start the REPL
mix zixir.repl
```

You should see:

```
Welcome to Zixir REPL v1.0.0
Type :help for help, :quit to exit

zixir>
```

Understanding the REPL

The REPL (Read-Eval-Print Loop) is your interactive playground. Let's explore:

```
zixir> 42
42

zixir> "Hello, Zixir!"
"Hello, Zixir!"

zixir> :help
REPL Commands:
  :help        - Show this help message
  :quit, :q    - Exit the REPL
  :clear       - Clear the screen
  :vars        - Show defined variables
  :engine      - List available engine operations
  :reset       - Clear all variables

zixir> :quit
Goodbye!
```

---

3. Your First Zixir Program

Hello World

Type this in the REPL:

```
zixir> "Hello, World!"
"Hello, World!"
```

What just happened?

1. You typed a string literal
2. Zixir evaluated it
3. It returned the string (which is also the value)
4. The REPL printed the result

Comments

Add explanations to your code:

```
# This is a comment - Zixir ignores it
let greeting = "Hello"  # Comments can be at the end of lines too
```

Your First Calculation

```
zixir> 10 + 20
30

zixir> 100 / 4
25.0

zixir> 5 * 5
25
```

Exercise 1: Try These

Type these in the REPL and observe the results:

```
1. 15 + 27
2. 100 - 33
3. 7 * 8
4. 81 / 9
5. "Hello" (just a string)
```

Questions to think about:

- What type of number do you get from division? (Hint: It has a decimal)
- What happens when you just type a string?

---

4. Understanding Variables and Types

What Are Variables?

Variables are like labeled boxes that store values. In Zixir, we use `let` to create a variable:

```
let age = 25           # A box labeled "age" containing 25
let name = "Alice"     # A box labeled "name" containing "Alice"
let pi = 3.14159       # A box labeled "pi" containing 3.14159
```

Important: Once you put a value in the box with `let`, you can't change it. This is called **immutability**.

Why Immutability?

Immutability makes your code safer and easier to understand:

```
# Without immutability (other languages)
let x = 5
x = 10  # Wait, what was x before? Did something else change it?

# With Zixir
let x = 5
# x will ALWAYS be 5 in this scope
```

```
# If you need a new value, create a new variable
let new_x = 10
```

Types in Zixir

Zixir has several built-in types:

1. Integers (Int)

Whole numbers without decimals:

```
let age = 25
let year = 2024
let negative = -10
let big_number = 1000000
```

2. Floats (Float)

Numbers with decimal points:

```
let pi = 3.14159
let price = 19.99
let temperature = -5.5
let scientific = 1.5e10  # 1.5 × 10^10 = 15000000000
```

3. Strings (String)

Text enclosed in double quotes:

```
let greeting = "Hello, World!"
let name = "Alice"
let empty = ""
let with_numbers = "Room 101"
```

String operations:

```
let first = "Hello"
let second = "World"
let combined = first ++ " " ++ second  # "Hello World"
```

4. Booleans (Bool)

True or false values:

```
let is_valid = true
let is_complete = false
```

Used for conditions and logic.

5. Arrays (Arrays)

Ordered collections of values:

```
let numbers = [1, 2, 3, 4, 5]
let names = ["Alice", "Bob", "Charlie"]
let mixed = [1, "two", 3.0, true]  # Can mix types
let empty = []
```

Accessing array elements:

```
let fruits = ["apple", "banana", "cherry"]
let first = fruits[0]    # "apple" (arrays start at 0)
let second = fruits[1]   # "banana"
let last = fruits[2]     # "cherry"
```

6. Maps (Maps)

Key-value pairs (like dictionaries):

```
let person = {
  "name": "Alice",
  "age": 30,
  "city": "New York"
}

let name = person["name"]   # "Alice"
let age = person["age"]     # 30
```

Type Annotations

While Zixir figures out types automatically, you can add annotations for clarity:

```
let age: Int = 25
let name: String = "Alice"
let price: Float = 19.99
let items: [Int] = [1, 2, 3, 4, 5]
```

When to use annotations:

- Function parameters and return types (recommended)
- Complex data structures
- When you want to be explicit about types

Constants

For values that never change (like mathematical constants):

```
const PI = 3.14159
const MAX_USERS = 100
const APP_NAME = "MyApp"
```

Constants are evaluated at compile time for better performance.

Exercise 2: Variables Practice

```
# 1. Create variables for:
#    - Your name (String)
#    - Your age (Int)
#    - Your height in meters (Float)
#    - Whether you like programming (Bool)

# 2. Create an array of your favorite foods

# 3. Create a map representing a book with title, author, and year

# 4. Try to change a variable (it should fail!)
#    let x = 5
#    let x = 10  # Error!
```

Common Mistakes

Mistake 1: Using single quotes

```
let wrong = 'hello'   # Error! Use double quotes
let right = "hello"   # Correct
```

Mistake 2: Forgetting quotes around strings

```
let wrong = hello     # Error! Zixir thinks hello is a variable
let right = "hello"   # Correct
```

Mistake 3: Array index out of bounds

```
let arr = [1, 2, 3]
let x = arr[10]       # Error! Index 10 doesn't exist (only 0, 1, 2)
```

---

5. Working with Data

Arrays in Detail

Arrays are ordered lists that can grow or shrink.

Creating arrays:

```
let numbers = [10, 20, 30, 40, 50]
let empty = []
let nested = [[1, 2], [3, 4], [5, 6]]  # Array of arrays
```

Getting the length:

```
let items = ["a", "b", "c"]
let count = length(items)  # 3
```

Working with ranges:

```
let range = 1..5  # Creates [1, 2, 3, 4, 5]
```

Common array patterns:

```
# Get first and last
let arr = [10, 20, 30, 40, 50]
let first = arr[0]
let last = arr[length(arr) - 1]

# Slice (subset of array)
let middle = [arr[1], arr[2], arr[3]]  # [20, 30, 40]
```

Maps in Detail

Maps store data as key-value pairs.

Creating maps:

```
let user = {
  "id": 1,
  "username": "alice",
  "email": "alice@example.com",
  "active": true
}
```

Accessing values:

```
let username = user["username"]  # "alice"
let email = user["email"]        # "alice@example.com"
```

Nested maps:

```
let company = {
  "name": "TechCorp",
  "address": {
    "street": "123 Main St",
    "city": "New York",
    "zip": "10001"
  },
  "employees": 50
}

let city = company["address"]["city"]  # "New York"
```

Checking if a key exists:

```
# Try to access - returns null if not found
let phone = user["phone"]  # null (doesn't exist)
```

Type Conversions

Sometimes you need to convert between types:

```
# String to number
let n = parse_int("42")      # 42 (Int)
let f = parse_float("3.14")  # 3.14 (Float)

# Number to string
let s = to_string(42)        # "42" (String)
let s2 = to_string(3.14)     # "3.14" (String)

# Array to string
let arr_str = to_string([1, 2, 3])  # "[1, 2, 3]"
```

Real-World Example: User Profile

```
let user = {
  "name": "Alice Johnson",
  "age": 28,
  "email": "alice@example.com",
  "skills": ["Python", "Zixir", "Machine Learning"],
  "is_active": true
}

# Extract information
let name = user["name"]
let email = user["email"]
let skill_count = length(user["skills"])
let first_skill = user["skills"][0]

# Create a summary
"User " ++ name ++ " has " ++ to_string(skill_count) ++ " skills"
# Result: "User Alice Johnson has 3 skills"
```

Exercise 3: Data Structures

```
# 1. Create an array of temperatures for a week: [72, 75, 68, 70, 74, 76, 73]
#    Calculate the average temperature

# 2. Create a map representing a product with:
#    - name (String)
#    - price (Float)
#    - in_stock (Bool)
```

```
#    - tags (Array of Strings)

# 3. Create a nested map for a library system:
#    Library has name, books (array of book maps)
#    Each book has title, author, year
```

---

6. Operators and Expressions

Arithmetic Operators

Operator	Description	Example	Result
`+`	Addition	`5 + 3`	`8`
`-`	Subtraction	`10 - 4`	`6`
`*`	Multiplication	`6 * 7`	`42`
`/`	Division	`15 / 3`	`5.0`
`-`	Negation	`-5`	`-5`

Important: Division always returns a Float, even if the result is a whole number.

```
10 / 2      # 5.0 (Float)
10 / 3      # 3.3333333333 (Float)
```

Comparison Operators

Operator	Description	Example	Result
`==`	Equal to	`5 == 5`	`true`
`!=`	Not equal to	`5 != 3`	`true`
`<`	Less than	`3 < 5`	`true`
`>`	Greater than	`7 > 4`	`true`
`<=`	Less than or equal	`5 <= 5`	`true`
`>=`	Greater than or equal	`6 >= 5`	`true`

```
let age = 25
let is_adult = age >= 18      # true
let is_senior = age >= 65     # false
let is_exactly_25 = age == 25 # true
```

Logical Operators

Operator	Description	Example	Result

`&&`	And	`true && false`	`false`
`` ` ``		`` ` ``	Or
`!`	Not	`!true`	`false`

```
let is_weekend = true
let is_sunny = true
let can_go_park = is_weekend && is_sunny  # true

let has_coffee = true
let has_tea = false
let has_drink = has_coffee || has_tea  # true

let is_raining = true
let not_raining = !is_raining  # false
```

Truth table: ``` A B A && B A || B !A

true true true true false true false false true false false true false true true false false false false true

```
### String Concatenation

```zixir
let first = "Hello"
let second = "World"
let combined = first ++ " " ++ second  # "Hello World"

let name = "Alice"
let greeting = "Hello, " ++ name ++ "!"  # "Hello, Alice!"
```

Operator Precedence

Just like in math, some operations happen before others:

```
1. Parentheses: ( )
2. Unary: -, !
3. Multiplication/Division: *, /
4. Addition/Subtraction: +, -
5. Comparison: <, >, <=, >=
6. Equality: ==, !=
7. Logical AND: &&
8. Logical OR: ||
```

```
# Without parentheses
5 + 3 * 2      # 11 (not 16) - multiplication first

# With parentheses
(5 + 3) * 2    # 16 - addition first
```
```

```
# Complex expression
10 + 5 * 2 > 15 && 4 < 8
# Step 1: 5 * 2 = 10
# Step 2: 10 + 10 = 20
# Step 3: 20 > 15 = true
# Step 4: 4 < 8 = true
# Step 5: true && true = true
```

Complex Expressions

```
# Calculate final price with tax
let price = 100.0
let tax_rate = 0.08
let discount = 10.0

let subtotal = price - discount
let tax = subtotal * tax_rate
let total = subtotal + tax

# Calculate average
let scores = [85, 92, 78, 96, 88]
let sum = scores[0] + scores[1] + scores[2] + scores[3] + scores[4]
let average = sum / 5

# Check if a number is in range
let x = 50
let min = 10
let max = 100
let in_range = x >= min && x <= max  # true
```

Exercise 4: Operators

```
# 1. Calculate the area of a rectangle (length * width)
#    length = 10, width = 5

# 2. Check if a temperature is comfortable (between 68 and 72)
#    temp = 70

# 3. Create a boolean expression for:
#    "User can access if they are admin OR (logged_in AND has_permission)"
#    is_admin = false
#    is_logged_in = true
#    has_permission = true

# 4. String concatenation:
#    Create the sentence "The temperature is 72 degrees"
#    from "The temperature is ", 72, and " degrees"
```

---

7. Control Flow

Control flow lets your program make decisions and repeat actions.

If/Else Expressions

The simplest form of decision-making:

```
if condition: result_if_true else: result_if_false
```

Basic example:

```
let age = 20
let status = if age >= 18: "adult" else: "minor"
# Result: "adult"
```

Without else:

```
let x = 10
let message = if x > 5: "big"
# Result: "big"

let y = 3
let message2 = if y > 5: "big"
# Result: null (nothing returned when condition is false and no else)
```

Multiple conditions with nesting:

```
let score = 85

let grade = if score >= 90: "A"
  else: if score >= 80: "B"
  else: if score >= 70: "C"
  else: if score >= 60: "D"
  else: "F"
# Result: "B"
```

Using logical operators:

```
let temp = 75
let is_sunny = true

let activity = if temp > 70 && is_sunny: "go to beach"
  else: if temp > 70: "go for walk"
  else: "stay inside"
```

While Loops

Repeat while a condition is true:

```
while condition: {
  # code to repeat
}
```

Count to 5:

```
let count = 0
while count < 5: {
  count = count + 1
}
count  # 5
```

Countdown:

```
let countdown = 10
while countdown > 0: {
  countdown = countdown - 1
}
countdown  # 0
```

Processing until done:

```
let data = [10, 20, 30, 40, 50]
let index = 0
let sum = 0

while index < length(data): {
  sum = sum + data[index]
  index = index + 1
}
sum  # 150
```

Warning: Make sure your loop will eventually end!

```
# BAD - infinite loop!
let x = 0
while x < 5: {
  # Forgot to update x!
  # This will run forever
}
```

For Loops

Iterate over each element in a collection:

```
for item in collection: {
  # code for each item
}
```

Basic iteration:

```
let fruits = ["apple", "banana", "cherry"]

for fruit in fruits: {
  # First: fruit = "apple"
  # Second: fruit = "banana"
  # Third: fruit = "cherry"
}
```

Calculating sum:

```
let numbers = [10, 20, 30, 40, 50]
let total = 0

for num in numbers: {
  total = total + num
}
total  # 150
```

Building a new array:

```
let numbers = [1, 2, 3, 4, 5]
let doubled = []

for num in numbers: {
  doubled = doubled ++ [num * 2]
}
doubled  # [2, 4, 6, 8, 10]
```

Working with maps:

```
let scores = {
  "Alice": 95,
  "Bob": 87,
  "Charlie": 92
}

# Note: For map iteration, we'd need engine operations
```

Real-World Example: Data Processing

```
# Calculate average temperature for the week
let temperatures = [72, 75, 68, 70, 74, 76, 73]
let sum = 0
let count = 0

for temp in temperatures: {
  sum = sum + temp
```

```
  count = count + 1
}

let average = sum / count  # 72.571428...

# Find days above average
let hot_days = 0
for temp in temperatures: {
  if temp > average: {
    hot_days = hot_days + 1
  }
}
hot_days  # 3 days were above average
```

Exercise 5: Control Flow

```
# 1. FizzBuzz (Classic programming problem)
#    Write a program that prints numbers 1 to 20, but:
#    - If divisible by 3, print "Fizz"
#    - If divisible by 5, print "Buzz"
#    - If divisible by both, print "FizzBuzz"
#    - Otherwise, print the number
#    Hint: Use modulo operator (num % 3 == 0 means divisible by 3)

# 2. Calculate factorial of 5 (5! = 5*4*3*2*1 = 120)
#    Use a while loop

# 3. Find the maximum value in an array [45, 12, 78, 23, 67, 89, 34]
#    Use a for loop

# 4. Count how many words in an array start with "a"
#    words = ["apple", "banana", "avocado", "cherry", "apricot"]
```

---

8. Functions

Functions are reusable blocks of code that perform specific tasks.

Why Use Functions?

Without functions (repetitive):

```
# Calculate area of three rectangles
let area1 = 10 * 5
let area2 = 8 * 4
let area3 = 12 * 6
```

With functions (clean and reusable):

```
fn rectangle_area(width, height): width * height

let area1 = rectangle_area(10, 5)
let area2 = rectangle_area(8, 4)
let area3 = rectangle_area(12, 6)
```

Defining Functions

Basic syntax:

```
fn function_name(param1, param2, ...): expression
```

Example:

```
fn greet(name): "Hello, " ++ name ++ "!"

# Call it
let message = greet("Alice")  # "Hello, Alice!"
```

With type annotations (recommended):

```
fn add(a: Int, b: Int) -> Int: a + b

fn greet(name: String) -> String: "Hello, " ++ name

fn is_adult(age: Int) -> Bool: age >= 18
```

With block body (multiple statements):

```
fn calculate_bmi(weight: Float, height: Float) -> Float: {
  let height_squared = height * height
  weight / height_squared
}
```

Calling Functions

```
fn add(a: Int, b: Int) -> Int: a + b
fn multiply(a: Int, b: Int) -> Int: a * b

# Call them
let sum = add(5, 3)           # 8
let product = multiply(4, 7)  # 28

# Chain function calls
let result = add(multiply(2, 3), 4)  # 10
# Step 1: multiply(2, 3) = 6
# Step 2: add(6, 4) = 10
```

Function Scope

Variables inside a function are separate from outside:

```
let x = 10  # Outside variable

fn double(n: Int) -> Int: {
  let x = n * 2  # This is a DIFFERENT x
  x
}

let result = double(5)  # 10
# x outside is still 10
```

Parameters vs Arguments

- **Parameters**: Variables in function definition
- **Arguments**: Actual values passed when calling

```
fn greet(name: String, greeting: String) -> String: {
  // name and greeting are PARAMETERS
  greeting ++ ", " ++ name ++ "!"
}

// "Alice" and "Hello" are ARGUMENTS
let msg = greet("Alice", "Hello")
```

Anonymous Functions (Lambdas)

Functions without names, useful for quick operations:

```
let double = fn(x: Int) -> Int: x * 2
let triple = fn(x: Int) -> Int: x * 3

double(5)  # 10
triple(5)  # 15
```

Recursion

Functions that call themselves:

```
fn factorial(n: Int) -> Int: {
  if n <= 1: 1
  else: n * factorial(n - 1)
}

factorial(5)  # 120
# 5 * 4 * 3 * 2 * 1 = 120
```

How it works:

```
factorial(5)
= 5 * factorial(4)
= 5 * (4 * factorial(3))
= 5 * (4 * (3 * factorial(2)))
= 5 * (4 * (3 * (2 * factorial(1))))
= 5 * (4 * (3 * (2 * 1)))
= 120
```

Public Functions

Mark functions that should be accessible from outside:

```
// Public function - can be called from other modules
pub fn public_api() -> Int: 42

// Private function (default) - only for internal use
fn helper() -> Int: 10
```

Real-World Example: Math Utilities

```
fn square(x: Float) -> Float: x * x
fn cube(x: Float) -> Float: x * x * x
fn average(a: Float, b: Float) -> Float: (a + b) / 2
fn percentage(part: Float, whole: Float) -> Float: (part / whole) * 100

// Use them
let radius = 5.0
let area = 3.14159 * square(radius)  # ~78.54

let score = 85.0
let max_score = 100.0
let pct = percentage(score, max_score)  # 85.0
```

Exercise 6: Functions

```
# 1. Write a function that converts Celsius to Fahrenheit
#    Formula: F = (C * 9/5) + 32

# 2. Write a function that checks if a number is even
#    Hint: Use modulo (num % 2 == 0)

# 3. Write a function that calculates the area of a circle
#    Formula: π * r²
#    Use const PI = 3.14159

# 4. Write a recursive function to calculate the sum of an array
#    fn array_sum(arr: [Int]) -> Int
```

```
# 5. Write a function that takes a name and age and returns
#    a greeting like "Hello Alice, you are 25 years old"
```

---

9. Pattern Matching

Pattern matching is one of Zixir's most powerful features. It lets you destructure data and make decisions based on its shape.

Basic Pattern Matching

```
match value {
  pattern1 => result1,
  pattern2 => result2,
  _ => default_result
}
```

Literal Patterns

Match exact values:

```
let x = 3

let word = match x {
  1 => "one",
  2 => "two",
  3 => "three",
  _ => "other"
}
// word = "three"
```

Variable Patterns

Capture the matched value:

```
let value = 42

let result = match value {
  0 => "zero",
  n => "The number is " ++ to_string(n)  // n captures the value
}
// result = "The number is 42"
```

The Wildcard Pattern `_`

Matches anything, used as a catch-all:

```
let x = 100

let category = match x {
```

```
  1 => "first",
  2 => "second",
  3 => "third",
  _ => "other"  // Matches anything else
}
// category = "other"
```

Array Patterns

Match arrays and extract elements:

```
let point = [3, 4]

let description = match point {
  [0, 0] => "origin",
  [x, 0] => "on x-axis",
  [0, y] => "on y-axis",
  [x, y] => "point at (" ++ to_string(x) ++ ", " ++ to_string(y) ++ ")"
}
// description = "point at (3, 4)"
```

Guards

Add conditions to patterns:

```
let age = 25

let category = match age {
  n if n < 0 => "invalid",
  n if n < 13 => "child",
  n if n < 20 => "teenager",
  n if n < 65 => "adult",
  _ => "senior"
}
// category = "adult"
```

Real-World Example: HTTP Status Codes

```
fn get_status_message(code: Int) -> String: {
  match code {
    200 => "OK",
    201 => "Created",
    400 => "Bad Request",
    401 => "Unauthorized",
    403 => "Forbidden",
    404 => "Not Found",
    500 => "Internal Server Error",
    c if c >= 200 && c < 300 => "Success",
    c if c >= 400 && c < 500 => "Client Error",
    c if c >= 500 && c < 600 => "Server Error",
```

```
    _ => "Unknown Status"
  }
}

get_status_message(404)  # "Not Found"
get_status_message(418)  # "Client Error"
```

Real-World Example: Command Processing

```
fn process_command(command: String) -> String: {
  match command {
    "start" => "Starting system...",
    "stop" => "Stopping system...",
    "restart" => "Restarting system...",
    "status" => "System is running",
    cmd => "Unknown command: " ++ cmd
  }
}

process_command("start")    # "Starting system..."
process_command("pause")    # "Unknown command: pause"
```

Exercise 7: Pattern Matching

```
# 1. Create a function that converts a number 1-7 to a day name
#    1 => "Monday", 2 => "Tuesday", etc.
#    Use _ for invalid numbers

# 2. Create a function that categorizes a temperature:
#    < 32 => "freezing"
#    32-50 => "cold"
#    51-70 => "mild"
#    71-85 => "warm"
#    > 85 => "hot"

# 3. Match on arrays to determine shape:
#    [] => "empty"
#    [x] => "single element"
#    [x, y] => "pair"
#    [x, y, z] => "triple"
#    _ => "many elements"

# 4. Create a simple calculator using pattern matching:
#    calculate(op, a, b) where op is "+", "-", "*", "/"
```

---

10. Engine Operations (Zig NIFs)

This is where Zixir shines! Engine operations are high-performance functions written in Zig that run 10-100x faster than equivalent operations in pure Elixir or Python.

What Are Engine Operations?

Think of them as super-fast built-in functions optimized for:

- Mathematical calculations
- Data processing
- Vector and matrix operations
- String manipulation

List Operations

Sum:

```
let numbers = [1.0, 2.0, 3.0, 4.0, 5.0]
let total = engine.list_sum(numbers)  # 15.0
```

Product:

```
let numbers = [2.0, 3.0, 4.0]
let product = engine.list_product(numbers)  # 24.0
```

Mean (Average):

```
let scores = [85.0, 92.0, 78.0, 96.0, 88.0]
let avg = engine.list_mean(scores)  # 87.8
```

Min and Max:

```
let values = [45.0, 23.0, 89.0, 12.0, 67.0]
let minimum = engine.list_min(values)  # 12.0
let maximum = engine.list_max(values)  # 89.0
```

Variance and Standard Deviation:

```
let data = [2.0, 4.0, 4.0, 4.0, 5.0, 5.0, 7.0, 9.0]
let var = engine.list_variance(data)  # 4.0
let std = engine.list_std(data)        # 2.0
```

Vector Operations

Dot Product:

```
let a = [1.0, 2.0, 3.0]
let b = [4.0, 5.0, 6.0]
let dot = engine.dot_product(a, b)  # 32.0 (1*4 + 2*5 + 3*6)
```

Vector Addition:

```
let a = [1.0, 2.0, 3.0]
let b = [10.0, 20.0, 30.0]
let sum = engine.vec_add(a, b)  # [11.0, 22.0, 33.0]
```

Vector Subtraction:

```
let a = [10.0, 20.0, 30.0]
let b = [1.0, 2.0, 3.0]
let diff = engine.vec_sub(a, b)  # [9.0, 18.0, 27.0]
```

Vector Scaling:

```
let v = [1.0, 2.0, 3.0]
let scaled = engine.vec_scale(v, 2.0)  # [2.0, 4.0, 6.0]
```

Matrix Operations

Matrix Multiplication:

```
let a = [[1.0, 2.0], [3.0, 4.0]]
let b = [[5.0, 6.0], [7.0, 8.0]]
let result = engine.mat_mul(a, b)  # [[19.0, 22.0], [43.0, 50.0]]
```

Transpose:

```
let m = [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]]
let transposed = engine.mat_transpose(m)  # [[1.0, 4.0], [2.0, 5.0], [3.0, 6.0]]
```

String Operations

String Length:

```
let text = "Hello, World!"
let len = engine.string_count(text)  # 13
```

Find Substring:

```
let text = "Hello, World!"
let pos = engine.string_find(text, "World")  # 7
```

Starts/Ends With:

```
let text = "Hello, World!"
let starts = engine.string_starts_with(text, "Hello")  # true
let ends = engine.string_ends_with(text, "World!")      # true
```

Transform Operations

Map Operations:

```
let numbers = [1.0, 2.0, 3.0, 4.0, 5.0]

let added = engine.map_add(numbers, 10.0)   # [11.0, 12.0, 13.0, 14.0, 15.0]
let multiplied = engine.map_mul(numbers, 2.0)  # [2.0, 4.0, 6.0, 8.0, 10.0]
```

Filter:

```
let numbers = [1.0, 5.0, 2.0, 8.0, 3.0, 9.0]
let big_numbers = engine.filter_gt(numbers, 4.0)  # [5.0, 8.0, 9.0]
```

Sort:

```
let messy = [5.0, 2.0, 8.0, 1.0, 9.0, 3.0]
let sorted = engine.sort_asc(messy)  # [1.0, 2.0, 3.0, 5.0, 8.0, 9.0]
```

Complete Engine Operations Reference

Operation	Description	Example
`engine.list_sum`	Sum of all elements	`engine.list_sum([1.0, 2.0])` → `3.0`
`engine.list_product`	Product of all elements	`engine.list_product([2.0, 3.0])` → `6.0`
`engine.list_mean`	Arithmetic mean	`engine.list_mean([2.0, 4.0])` → `3.0`
`engine.list_min`	Minimum value	`engine.list_min([3.0, 1.0])` → `1.0`
`engine.list_max`	Maximum value	`engine.list_max([3.0, 1.0])` → `3.0`
`engine.list_variance`	Statistical variance	`engine.list_variance(data)`
`engine.list_std`	Standard deviation	`engine.list_std(data)`
`engine.dot_product`	Dot product of two vectors	`engine.dot_product(a, b)`
`engine.vec_add`	Element-wise addition	`engine.vec_add(a, b)`
`engine.vec_sub`	Element-wise subtraction	`engine.vec_sub(a, b)`
`engine.vec_mul`	Element-wise multiplication	`engine.vec_mul(a, b)`
`engine.vec_div`	Element-wise division	`engine.vec_div(a, b)`
`engine.vec_scale`	Scale vector by scalar	`engine.vec_scale(v, 2.0)`
`engine.mat_mul`	Matrix multiplication	`engine.mat_mul(a, b)`
`engine.mat_transpose`	Matrix transpose	`engine.mat_transpose(m)`
`engine.string_count`	String length in bytes	`engine.string_count(s)`

`engine.string_find`	Find substring index	`engine.string_find(s, sub)`
`engine.string_starts_with`	Check prefix	`engine.string_starts_with(s, prefix)`
`engine.string_ends_with`	Check suffix	`engine.string_ends_with(s, suffix)`
`engine.map_add`	Add value to each element	`engine.map_add(arr, val)`
`engine.map_mul`	Multiply each element	`engine.map_mul(arr, val)`
`engine.filter_gt`	Filter elements > value	`engine.filter_gt(arr, val)`
`engine.sort_asc`	Sort ascending	`engine.sort_asc(arr)`

Exercise 8: Engine Operations

```
# 1. Calculate statistics for this dataset:
#    data = [23.5, 25.0, 22.8, 24.2, 26.1, 23.9, 25.5]
#    Calculate: sum, mean, min, max, std

# 2. Normalize this array (subtract mean, divide by std):
#    data = [10.0, 20.0, 30.0, 40.0, 50.0]
#    Hint: Use engine operations

# 3. Find the cosine similarity between two vectors:
#    Formula: dot(a, b) / (sqrt(dot(a, a)) * sqrt(dot(b, b)))
#    a = [1.0, 2.0, 3.0]
#    b = [4.0, 5.0, 6.0]
```

---

11. Python Integration

Zixir can call any Python library, giving you access to the world's largest collection of software.

Basic Python Calls

Syntax:

```
python "module_name" "function_name" (arguments)
```

Example - Math operations:

```
let sqrt = python "math" "sqrt" (16.0)      # 4.0
let power = python "math" "pow" (2.0, 3.0)  # 8.0
let pi = python "math" "pi" ()              # 3.141592653589793
```

Example - JSON parsing:

```
let json_text = "{\"name\": \"Alice\", \"age\": 30}"
let data = python "json" "loads" (json_text)
```

```
# data = {"name": "Alice", "age": 30}
```

Example - Statistics:

```
let numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
let mean = python "statistics" "mean" (numbers)      # 5.5
let median = python "statistics" "median" (numbers)  # 5.5
let stdev = python "statistics" "stdev" (numbers)    # 3.027...
```

Passing Arguments

Simple arguments:

```
python "math" "sqrt" (16.0)  # Single argument
python "math" "pow" (2.0, 3.0)  # Multiple arguments
```

Arrays as arguments:

```
let data = [1, 2, 3, 4, 5]
let total = python "sum" (data)  # 15
```

Common Python Libraries

Math and Statistics:

```
python "math" "sin" (3.14159 / 2)        # Trigonometry
python "math" "log" (100.0)              # Natural log
python "math" "factorial" (5)            # 120
python "random" "random" ()              # Random number 0-1
python "random" "randint" (1, 100)       # Random int 1-100
```

Data Processing:

```
# Sorting
let arr = [5, 2, 8, 1, 9]
let sorted = python "sorted" (arr)  # [1, 2, 5, 8, 9]

# String manipulation
let text = "hello world"
let upper = python "str.upper" (text)  # "HELLO WORLD"
```

Date and Time:

```
let now = python "datetime.datetime" "now" ()
let today = python "datetime.date" "today" ()
```

When to Use Python vs Engine

Use Engine Operations when:

- Doing bulk math on arrays
- Need maximum performance
- Working with vectors/matrices
- Simple transformations

Use Python when:

- Need specific libraries (ML, NLP, etc.)
- Complex data processing
- JSON/XML parsing
- File operations
- External APIs

Error Handling

```
# Python errors become Zixir errors
try {
  let result = python "math" "sqrt" (-1.0)
} catch PythonError => {
  "Cannot calculate square root of negative number"
}
```

Exercise 9: Python Integration

```
# 1. Use Python's random module to:
#    - Generate a random number between 1 and 100
#    - Pick a random element from ["apple", "banana", "cherry"]

# 2. Parse this JSON string using Python:
#    json_str = "{\"users\": [{\"name\": \"Alice\", \"score\": 95}, {\"name\": \"Bob\",
\"score\": 87}]}"
#    Extract the list of user names

# 3. Use Python's datetime to get:
#    - Current date
#    - Current year
#    - Day of the week

# 4. Challenge: Use Python's base64 to encode/decode a string
```

---

12. Real-World Project 1: Data Processing Pipeline

Let's build a complete data processing system that:

1. Loads temperature data
2. Cleans and validates it
3. Calculates statistics
4. Identifies anomalies

5. Generates a report

The Complete Solution

```
# ============================================
# Data Processing Pipeline
# ============================================

# Step 1: Define our data
let raw_temperatures = [
  72.5, 73.0, 71.5, 74.2, 75.0,  // Monday
  76.1, 75.5, 74.8, 73.2, 72.5,  // Tuesday
  71.0, 70.5, 69.8, 71.2, 72.0,  // Wednesday
  999.0, 73.5, 74.0, 75.2, 76.5, // Thursday (has outlier!)
  77.0, 76.2, 75.8, 74.5, 73.5   // Friday
]

# Step 2: Clean the data (remove outliers)
fn clean_data(data: [Float], min: Float, max: Float) -> [Float]: {
  let cleaned = []
  for temp in data: {
    if temp >= min && temp <= max: {
      cleaned = cleaned ++ [temp]
    }
  }
  cleaned
}

let valid_range_min = 60.0
let valid_range_max = 90.0
let temperatures = clean_data(raw_temperatures, valid_range_min, valid_range_max)

# Step 3: Calculate statistics using engine
let stats = {
  "count": length(temperatures),
  "sum": engine.list_sum(temperatures),
  "mean": engine.list_mean(temperatures),
  "min": engine.list_min(temperatures),
  "max": engine.list_max(temperatures),
  "std": engine.list_std(temperatures)
}

# Step 4: Identify anomalies (values > 2 std from mean)
let threshold_high = stats["mean"] + (2.0 * stats["std"])
let threshold_low = stats["mean"] - (2.0 * stats["std"])

let anomalies_high = engine.filter_gt(temperatures, threshold_high)
let anomalies_low = []  // We filtered out low outliers, but here's the pattern

# Step 5: Generate report
let report = {
  "dataset_info": {
```

```
    "original_count": length(raw_temperatures),
    "cleaned_count": stats["count"],
    "outliers_removed": length(raw_temperatures) - stats["count"]
  },
  "statistics": stats,
  "temperature_range": {
    "min": stats["min"],
    "max": stats["max"],
    "span": stats["max"] - stats["min"]
  },
  "anomalies": {
    "high_threshold": threshold_high,
    "low_threshold": threshold_low,
    "high_anomalies": anomalies_high,
    "low_anomalies": anomalies_low
  }
}

# Output the report
report
```

Breaking It Down

Step 1 - Data Definition: We define an array of temperature readings over 5 days. Notice there's an outlier (999.0) that we'll need to remove.

Step 2 - Data Cleaning:

```
fn clean_data(data: [Float], min: Float, max: Float) -> [Float]: {
  let cleaned = []
  for temp in data: {
    if temp >= min && temp <= max: {
      cleaned = cleaned ++ [temp]
    }
  }
  cleaned
}
```

This function:

- Takes data and valid range
- Creates empty array for cleaned data
- Loops through each temperature
- Checks if it's within range
- If yes, adds to cleaned array
- Returns cleaned array

Step 3 - Statistics Calculation:

```
let stats = {
  "count": length(temperatures),
  "sum": engine.list_sum(temperatures),
```

```
  "mean": engine.list_mean(temperatures),
  "min": engine.list_min(temperatures),
  "max": engine.list_max(temperatures),
  "std": engine.list_std(temperatures)
}
```

We use Zixir's fast engine operations to calculate all statistics at once.

Step 4 - Anomaly Detection:

```
let threshold_high = stats["mean"] + (2.0 * stats["std"])
let threshold_low = stats["mean"] - (2.0 * stats["std"])
```

Standard statistical method: values more than 2 standard deviations from mean are anomalies.

Step 5 - Report Generation: We create a nested map with all our findings, organized logically.

Expected Output

```
{
  "dataset_info": {
    "original_count": 25,
    "cleaned_count": 24,
    "outliers_removed": 1
  },
  "statistics": {
    "count": 24,
    "sum": 1764.9,
    "mean": 73.5375,
    "min": 69.8,
    "max": 77.0,
    "std": 2.043...
  },
  "temperature_range": {
    "min": 69.8,
    "max": 77.0,
    "span": 7.2
  },
  "anomalies": {
    "high_threshold": 77.623...,
    "low_threshold": 69.451...,
    "high_anomalies": [],
    "low_anomalies": []
  }
}
```

What We Learned

1. **Data pipelines** are sequences of transformation steps
2. **Functions** make code reusable (clean_data)
3. **Engine operations** are perfect for bulk statistics

4. **Maps** organize complex results cleanly
5. **Error handling** (outlier removal) is essential

---

13. Real-World Project 2: AI Text Analysis

Build a text analysis system that:

1. Processes text data
2. Calculates basic statistics
3. Uses Python for tokenization
4. Calculates word frequencies
5. Generates insights

The Complete Solution

```
# ==========================================
# AI Text Analysis System
# ==========================================

# Sample text to analyze
let text = "The quick brown fox jumps over the lazy dog. The fox is quick and the dog is
lazy. Programming is fun and programming is useful."

# Step 1: Basic text statistics
let char_count = engine.string_count(text)
let sentences = 3  // We can count periods

// Step 2: Use Python to tokenize (split into words)
let words_str = python "str" "lower" (text)
let words = python "str" "split" (words_str)

// Step 3: Calculate word statistics
let word_count = length(words)
let unique_words = python "set" (words)
let unique_count = length(unique_words)

// Step 4: Calculate word frequencies using Python
let word_freq = {}
for word in words: {
  let current_count = word_freq[word]
  if current_count == null: {
    word_freq[word] = 1
  } else: {
    word_freq[word] = current_count + 1
  }
}

// Step 5: Find most common words
let max_freq = 0
let most_common = ""
for word in unique_words: {
```

```
  let freq = word_freq[word]
  if freq > max_freq: {
    max_freq = freq
    most_common = word
  }
}

// Step 6: Calculate average word length
let total_chars = 0
for word in words: {
  total_chars = total_chars + engine.string_count(word)
}
let avg_word_length = total_chars / word_count

// Step 7: Generate comprehensive report
let analysis = {
  "text_info": {
    "character_count": char_count,
    "word_count": word_count,
    "sentence_count": sentences,
    "unique_words": unique_count,
    "vocabulary_richness": unique_count / word_count
  },
  "word_stats": {
    "average_word_length": avg_word_length,
    "most_common_word": most_common,
    "most_common_frequency": max_freq
  },
  "word_frequency": word_freq,
  "insights": {
    "readability": if avg_word_length < 5: "easy" else: if avg_word_length < 7: "medium"
else: "hard",
    "vocabulary_diversity": if (unique_count / word_count) > 0.7: "high" else: "low"
  }
}

analysis
```

Key Features Demonstrated

1. **Text Processing**: Using both engine and Python operations
2. **Data Structures**: Arrays for words, maps for frequencies
3. **Looping**: Multiple for loops for different calculations
4. **Statistics**: Word counts, frequencies, averages
5. **Insights**: Derived metrics (readability, vocabulary diversity)

---

14. Real-World Project 3: LLM Integration

Integrate with OpenAI's API to build an intelligent assistant:

The Complete Solution

```
# ============================================
# LLM Integration with OpenAI
# ============================================

// Configuration
const OPENAI_API_KEY = "your-api-key-here"
const MODEL = "gpt-3.5-turbo"

// Step 1: Function to call OpenAI API
fn call_openai(prompt: String) -> String: {
  // Build the request
  let request = {
    "model": MODEL,
    "messages": [
      {"role": "system", "content": "You are a helpful assistant."},
      {"role": "user", "content": prompt}
    ],
    "max_tokens": 150,
    "temperature": 0.7
  }

  // Convert to JSON
  let json_request = python "json" "dumps" (request)

  // Make HTTP request (using Python requests library)
  let response = python "requests.post" (
    "https://api.openai.com/v1/chat/completions",
    {
      "headers": {
        "Authorization": "Bearer " ++ OPENAI_API_KEY,
        "Content-Type": "application/json"
      },
      "data": json_request
    }
  )

  // Parse response
  let result = python "json" "loads" (response.text)
  result["choices"][0]["message"]["content"]
}

// Step 2: Function to summarize text
fn summarize(text: String) -> String: {
  let prompt = "Summarize the following text in 2-3 sentences:\n\n" ++ text
  call_openai(prompt)
}

// Step 3: Function to extract entities
fn extract_entities(text: String) -> [String]: {
  let prompt = "Extract all named entities (people, places, organizations) from this text as
a comma-separated list:\n\n" ++ text
```

```
  let response = call_openai(prompt)
  python "str" "split" (response, ", ")
}

// Step 4: Function to classify sentiment
fn classify_sentiment(text: String) -> String: {
  let prompt = "Classify the sentiment of this text as 'positive', 'negative', or 'neutral'.
Only respond with one word.\n\n" ++ text
  call_openai(prompt)
}

// Step 5: Process a document
let document = "Apple Inc. announced record quarterly earnings today in Cupertino,
California. CEO Tim Cook expressed excitement about the new iPhone lineup and AI
initiatives. Investors responded positively to the news."

let results = {
  "original_text": document,
  "summary": summarize(document),
  "entities": extract_entities(document),
  "sentiment": classify_sentiment(document)
}

results
```

What This Demonstrates

1. **API Integration**: Calling external services
2. **JSON Handling**: Building and parsing JSON
3. **String Manipulation**: Building prompts
4. **Function Composition**: Chaining operations
5. **Error Handling**: Production-ready patterns

---

15. Real-World Project 4: Workflow Automation

Build a complete workflow system for processing orders:

The Complete Solution

```
# ============================================
# Order Processing Workflow
# ============================================

// Step 1: Define the workflow steps
fn validate_order(order: Map) -> Map: {
  let is_valid = order["amount"] > 0 &&
                 order["customer"] != "" &&
                 order["items"] != []

  if is_valid: {
    {"status": "valid", "order": order}
```

```
  } else: {
    {"status": "invalid", "error": "Order validation failed"}
  }
}

fn calculate_discount(order: Map) -> Map: {
  let amount = order["amount"]
  let discount_rate = if amount > 1000: 0.15
                      else: if amount > 500: 0.10
                      else: 0.05

  let discount = amount * discount_rate
  let final_amount = amount - discount

  {
    "discount_rate": discount_rate,
    "discount_amount": discount,
    "final_amount": final_amount
  }
}

fn check_inventory(items: [String]) -> Map: {
  // Simulate inventory check
  let available = ["laptop", "mouse", "keyboard"]
  let out_of_stock = []

  for item in items: {
    if !python "item in available": {
      out_of_stock = out_of_stock ++ [item]
    }
  }

  if out_of_stock == []: {
    {"status": "available", "items": items}
  } else: {
    {"status": "unavailable", "out_of_stock": out_of_stock}
  }
}

fn process_payment(amount: Float, method: String) -> Map: {
  // Simulate payment processing
  let success = if method == "credit_card" || method == "paypal": true
                else: false

  if success: {
    {
      "status": "success",
      "transaction_id": python "str(uuid.uuid4())" (),
      "amount": amount,
      "method": method
    }
  } else: {
```

```
    {"status": "failed", "error": "Payment method not supported"}
  }
}

fn send_notification(customer: String, message: String) -> String: {
  // Simulate sending email/SMS
  "Notification sent to " ++ customer ++ ": " ++ message
}

// Step 2: Main workflow orchestrator
fn process_order(order: Map) -> Map: {
  // Step 2.1: Validate
  let validation = validate_order(order)
  if validation["status"] != "valid": {
    return {
      "success": false,
      "stage": "validation",
      "error": validation["error"]
    }
  }

  // Step 2.2: Check inventory
  let inventory = check_inventory(order["items"])
  if inventory["status"] != "available": {
    return {
      "success": false,
      "stage": "inventory",
      "error": "Items out of stock: " ++ to_string(inventory["out_of_stock"])
    }
  }

  // Step 2.3: Calculate pricing
  let pricing = calculate_discount(order)

  // Step 2.4: Process payment
  let payment = process_payment(pricing["final_amount"], order["payment_method"])
  if payment["status"] != "success": {
    return {
      "success": false,
      "stage": "payment",
      "error": payment["error"]
    }
  }

  // Step 2.5: Send confirmation
  let notification = send_notification(
    order["customer"],
    "Order confirmed! Total: $" ++ to_string(pricing["final_amount"])
  )

  // Step 2.6: Return success
  {
```

```
    "success": true,
    "order_id": python "str(uuid.uuid4())" (),
    "original_amount": order["amount"],
    "discount": pricing["discount_amount"],
    "final_amount": pricing["final_amount"],
    "transaction_id": payment["transaction_id"],
    "notification": notification
  }
}

// Step 3: Test the workflow
let test_order = {
  "customer": "alice@example.com",
  "items": ["laptop", "mouse"],
  "amount": 1200.00,
  "payment_method": "credit_card"
}

process_order(test_order)
```

Workflow Pattern Explanation

This demonstrates the **Pipeline Pattern**:

1. **Validate** → Check if order is valid
2. **Inventory** → Check if items are available
3. **Calculate** → Apply discounts
4. **Payment** → Process payment
5. **Notify** → Send confirmation

Each step:

- Returns a map with status
- Can short-circuit on failure
- Passes data to next step
- Logs what happened

---

16. Best Practices

Code Organization

Good:

```
// Group related functions
fn validate(data) -> Bool
fn transform(data) -> Data
fn save(data) -> Result

// Use clear variable names
let customer_email = "alice@example.com"
let total_price = calculate_total(items)
```

Bad:

```
// Unclear naming
fn f1(x)
fn f2(y)
let a = 123
let b = "test"
```

Performance Tips

1. **Use engine operations for bulk math**

   ```
   // Good - fast
   let sum = engine.list_sum(data)

   // Bad - slow
   let sum = 0
   for x in data: { sum = sum + x }
   ```

2. **Minimize Python FFI calls**

   ```
   // Good - batch operations
   let results = python "process_batch" (data)

   // Bad - individual calls
   for item in data: {
     python "process" (item)
   }
   ```

3. **Prefer pattern matching over nested ifs**

   ```
   // Good - clear intent
   match status {
     "success" => handle_success(),
     "error" => handle_error(),
     _ => handle_unknown()
   }

   // Bad - hard to read
   if status == "success": {
     handle_success()
   } else: {
     if status == "error": {
       handle_error()
     } else: {
       handle_unknown()
     }
   }
   ```

Error Handling

```
// Always handle errors explicitly
fn divide(a: Float, b: Float) -> Float: {
  if b == 0: {
    // Return error indication
    null  // Or use Result type pattern
  } else: {
    a / b
  }
}

// Use try/catch for external operations
try {
  let data = python "fetch_data" ()
} catch PythonError => {
  "Failed to fetch data"
}
```

Naming Conventions

- **Variables**: `snake_case` (`customer_name`, `total_amount`)
- **Functions**: `snake_case` (`calculate_total`, `process_order`)
- **Constants**: `SCREAMING_SNAKE_CASE` (`MAX_SIZE`, `API_KEY`)
- **Types**: `PascalCase` (`Int`, `String`, `MyType`)

Documentation

```
// Function documentation
fn calculate_area(width: Float, height: Float) -> Float: {
  // Calculates the area of a rectangle
  //
  // Args:
  //   width: The width of the rectangle
  //   height: The height of the rectangle
  //
  // Returns:
  //   The area (width * height)
  width * height
}
```

---

17. Debugging Guide

Reading Error Messages

Syntax Error:

```
Parse error: Unexpected token: 'let' at line 5, column 1
```

→ Check if previous statement is missing a closing brace or semicolon

Undefined Variable:

```
Error: Undefined variable: 'count'
```

→ Variable used before definition, or typo in name

Type Mismatch:

```
Error: Type mismatch: expected Int, got String
```

→ Passing wrong type to function or operator

Debugging Techniques

1. Print Debugging:

```
let x = calculate_something()
// Add temporary print
python "print" ("Debug: x = ", x)
```

2. REPL Testing: Test small pieces in the REPL before adding to your program.

3. Simplify: When you have a bug, comment out half your code to isolate the problem.

4. Check Types:

```
// Use Python to check types
python "type" (variable)
```

Common Mistakes

Mistake	Why It Happens	Solution
`Undefined variable`	Typo or using before defining	Check spelling and order
`Type mismatch`	Wrong type in operation	Add type annotations
`Parse error`	Missing brackets/parentheses	Check syntax carefully
`Array index out of bounds`	Accessing non-existent index	Check array length first
`Infinite loop`	Loop condition never becomes false	Ensure loop variable changes

---

18. Complete Grammar Reference

Statements

```
// Variable declaration
let name = expression
let name: Type = expression
const NAME = expression
```

```
// Expression statement
expression

// Control flow
if condition: expression else: expression
while condition: { block }
for item in collection: { block }

// Pattern matching
match value {
  pattern1 => result1,
  pattern2 => result2,
  _ => default
}

// Error handling
try {
  expression
} catch ErrorType => {
  handler
}
```

Expressions

```
// Literals
42           // Integer
3.14         // Float
"hello"      // String
true, false  // Boolean
[1, 2, 3]    // Array
{"a": 1}     // Map

// Variables
variable_name

// Function call
function_name(arg1, arg2)

// Engine call
engine.operation_name(arg1, arg2)

// Python call
python "module" "function" (args)

// Array access
array[index]

// Map access
map["key"]

// Field access
```

```
object.field

// Parentheses
(expression)
```

Operators

Operator	Precedence	Description
`()`	1	Parentheses
`[]`	1	Indexing
`.`	1	Field access
`- !`	2	Unary negation, not
`* /`	3	Multiplication, division
`+ -`	4	Addition, subtraction
`++`	4	Concatenation
`< > <= >=`	5	Comparison
`== !=`	6	Equality
`&&`	7	Logical AND
`` ` ``		`` ` ``

Function Definition

```
// Basic function
fn name(param1, param2): expression

// With types
fn name(param1: Type1, param2: Type2) -> ReturnType: expression

// With block
fn name(params): {
  statement1
  statement2
  expression  // Return value
}

// Public function
pub fn name(params): expression

// Anonymous function
fn(params): expression
```

Types

```
// Primitive types
Int      // Integer
Float    // Floating point
String   // Text
Bool     // Boolean

// Collection types
[Type]           // Array of Type
{String: Type}   // Map with string keys

// Function types
(Type1, Type2) -> ReturnType

// Type annotations
let x: Int = 5
let arr: [Float] = [1.0, 2.0]
fn add(a: Int, b: Int) -> Int: a + b
```

---

19. Quick Reference Card

Variables & Types

```
let x = 5                    // Immutable variable
let y: Int = 10             // With type annotation
const PI = 3.14159          // Compile-time constant
```

Operators

```
// Arithmetic
+  -  *  /

// Comparison
==  !=  <  >  <=  >=

// Logical
&&  ||  !

// Other
++   // String/array concatenation
[]   // Array indexing
```

Control Flow

```
// If/else
if condition: expr else: expr

// While loop
```

```
while condition: { block }

// For loop
for item in list: { block }

// Pattern matching
match value {
  pattern => result,
  _ => default
}
```

Functions

```
// Named function
fn name(a: Int, b: Int) -> Int: a + b

// Anonymous function
fn(x): x * 2

// Call
name(5, 3)
```

Engine Operations

```
// Lists
engine.list_sum(arr)
engine.list_mean(arr)
engine.list_min(arr)
engine.list_max(arr)

// Vectors
engine.vec_add(a, b)
engine.vec_sub(a, b)
engine.dot_product(a, b)

// Transform
engine.map_add(arr, val)
engine.filter_gt(arr, val)
engine.sort_asc(arr)
```

Python FFI

```
python "math" "sqrt" (16.0)
python "json" "loads" (json_str)
python "random" "random" ()
```

Common Patterns

Loop with index:

```
let i = 0
while i < length(arr): {
  // use arr[i]
  i = i + 1
}
```

Filter array:

```
let filtered = []
for item in arr: {
  if condition: {
    filtered = filtered ++ [item]
  }
}
```

Map operation:

```
let result = []
for item in arr: {
  result = result ++ [transform(item)]
}
```

---

20. Appendix: Common Patterns

Pattern 1: Data Validation Pipeline

```
fn validate_email(email: String) -> Bool: {
  email != "" && engine.string_find(email, "@") >= 0
}

fn validate_age(age: Int) -> Bool: {
  age >= 0 && age <= 150
}

fn validate_user(user: Map) -> Map: {
  let errors = []

  if !validate_email(user["email"]): {
    errors = errors ++ ["Invalid email"]
  }

  if !validate_age(user["age"]): {
    errors = errors ++ ["Invalid age"]
  }

  if errors == []: {
```

```
    {"valid": true}
  } else: {
    {"valid": false, "errors": errors}
  }
}
```

Pattern 2: Retry Logic

```
fn retry_operation(operation: Function, max_attempts: Int) -> Result: {
  let attempts = 0
  let result = null

  while attempts < max_attempts && result == null: {
    try {
      result = operation()
    } catch Error => {
      attempts = attempts + 1
      if attempts < max_attempts: {
        // Wait before retry (using Python)
        python "time.sleep" (1.0)
      }
    }
  }

  if result == null: {
    {"success": false, "error": "Max retries exceeded"}
  } else: {
    {"success": true, "result": result}
  }
}
```

Pattern 3: Batch Processing

```
fn process_batch(items: [Item], batch_size: Int) -> [Result]: {
  let results = []
  let i = 0

  while i < length(items): {
    // Get batch
    let batch = []
    let j = 0
    while j < batch_size && (i + j) < length(items): {
      batch = batch ++ [items[i + j]]
      j = j + 1
    }

    // Process batch (using engine for speed)
    let batch_results = engine.process_batch(batch)
    results = results ++ batch_results
```

```
    i = i + batch_size
  }

  results
}
```

Pattern 4: Configuration Management

```
const DEFAULT_CONFIG = {
  "max_retries": 3,
  "timeout": 30,
  "batch_size": 100,
  "debug": false
}

fn load_config(overrides: Map) -> Map: {
  // Merge overrides with defaults
  let config = DEFAULT_CONFIG
  for key in python "overrides.keys" (): {
    config[key] = overrides[key]
  }
  config
}
```

Pattern 5: Result Type

```
// Simulating a Result type for error handling
fn safe_divide(a: Float, b: Float) -> Map: {
  if b == 0: {
    {"ok": false, "error": "Division by zero"}
  } else: {
    {"ok": true, "value": a / b}
  }
}

// Usage
let result = safe_divide(10.0, 2.0)
if result["ok"]: {
  // Use result["value"]
} else: {
  // Handle result["error"]
}
```

Conclusion

Congratulations! You've completed the **Zixir Language Complete Guide**. You now have:

✅ **Solid foundation** in Zixir syntax and concepts
✅ **Practical skills** for building real programs
✅ **Understanding** of Zixir's unique features (engine, Python FFI)
✅ **Experience** with real-world projects
✅ **Reference materials** for ongoing development

Next Steps

1. **Practice**: Complete the exercises in each chapter
2. **Build**: Create your own Zixir projects
3. **Share**: Join the Zixir community
4. **Contribute**: Help improve Zixir

Resources

- **Official Website**: https://zixir-lang.org
- **Documentation**: https://docs.zixir-lang.org
- **GitHub**: https://github.com/Zixir-lang/Zixir
- **Community Forum**: https://forum.zixir-lang.org

Keep Learning

- Experiment with the REPL
- Read the standard library source code
- Build increasingly complex projects
- Share what you learn with others

Happy coding with Zixir! 🚀

This guide was written for Zixir v1.0.0 Last updated: February 2024